

基于 Rust 语言的应用开发——Rougelike 游戏界面设计

一、实验目的

本次项目的核心目的在于验证和展示 Rust 与 ECS 架构在游戏逻辑开发中的可行性。探讨 Rust 中运用 ECS 架构如何进行游戏开发并且如何实现复杂系统以及游戏整体的可扩展性，提供一个基于 Rust 的 Rougelike 游戏系统。

二、实验环境

1. 开发工具：Visual Studio Code
2. 操作系统：Windows 11

三、实验原理

本项目的实验原理基于以下三个核心的技术理论，它们共同构成了高性能、可维护的游戏逻辑基础：

3.1 实体组件系统 (ECS) 原理

核心理论： 彻底的数据驱动与关注点分离。

原理应用： 将游戏对象解构为三个基本元素（实体 Entity、组件 Component、系统 System）。组件（纯数据）在内存中紧凑排列，实现数据局部性，极大提升 CPU 缓存命中率。系统（纯逻辑）仅查询和操作所需的组件子集，解耦逻辑间的依赖，方便独立维护和升级。

3.2 Rust 借用模型与并发安全原理

核心理论： Rust 的所有权和借用模型。

原理应用： Specs ECS 调度器在执行系统时，利用 Rust 的借用检查机制（Read/Write）在编译期或运行时强制保证任何时刻，对组件存储的访问要么是一个写入引用，要么是多个读取引用。使得系统能够安全地并行执行（例如：MonsterAI 写入怪物位置，RenderSystem 读取所有实体位置），从根本上消除了传统游戏引擎中常见的数据竞争问题。

3.3 程序化生成与空间算法原理

核心理论： 随机性约束和几何空间投射。

原理应用：

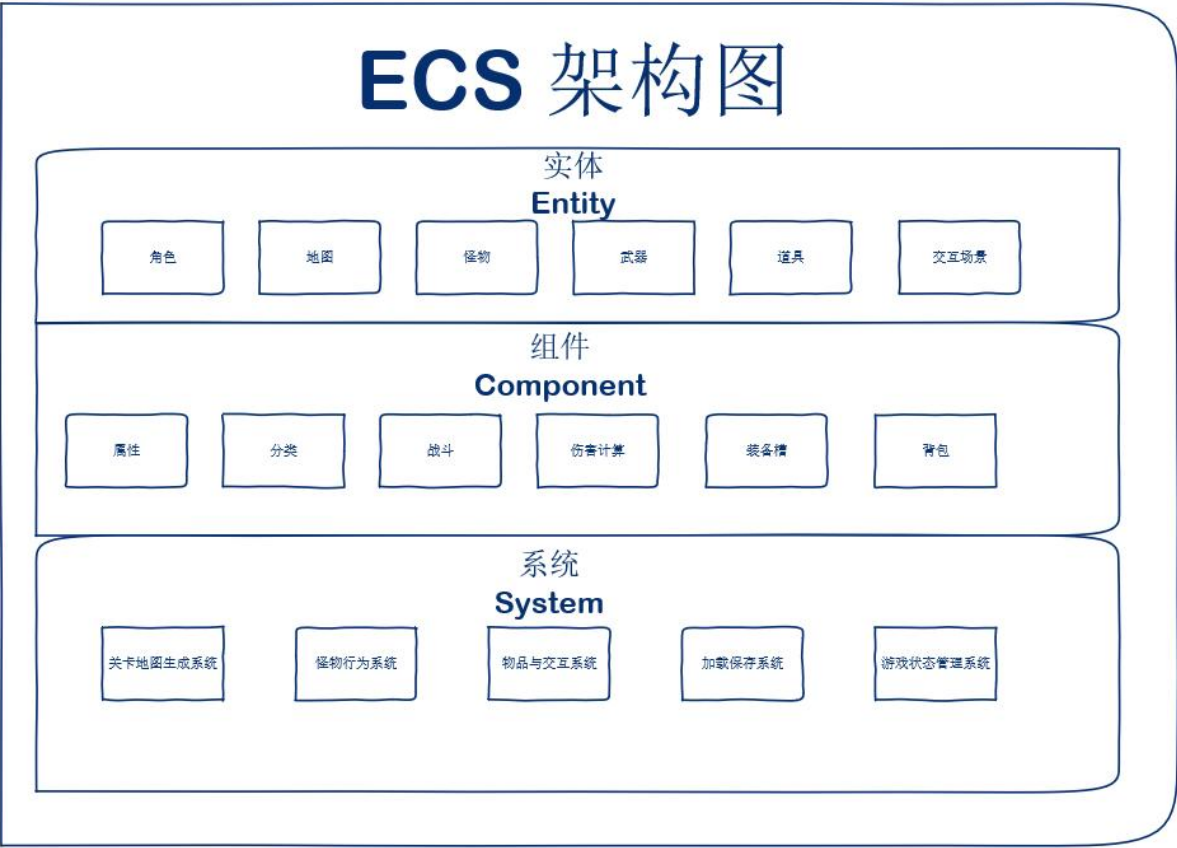
地牢生成： 采用二叉空间分割算法，确保生成的地图在随机的同时，具备结构完整性和可连接性，保证了游戏的每次体验都是独一无二但可玩的。

视野： 采用递归阴影投射算法，模拟真实世界的光线遮挡，以高性能的方式计算出玩家的可见区域。

四、实验步骤

（一）需求分析与设计规划

本实验项目的核心目标是开发一个功能完备的回合制地牢探险游戏（Rougelike），并采用现代游戏开发中高效的 实体-组件-系统（ECS） 架构。



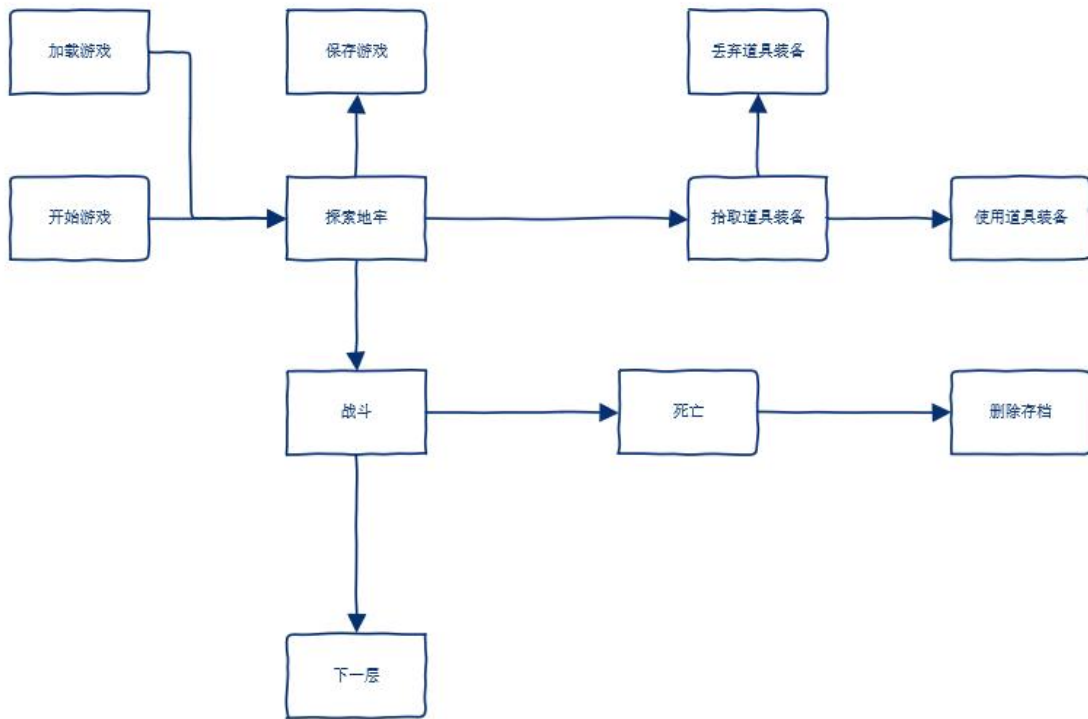
游戏的 ECS 架构设计图

4.1.1. 应用需求描述

模块	核心功能需求
地图/环境	1. 程序化生成地牢： 每次游戏都生成结构随机的地图（房间和走廊）。 2. 视野系统（FOV）： 玩家只能看到视野范围内的区域，增强探索感。
核心机制	1. 回合制： 严格遵循“玩家行动 → 怪物行动 → 渲染”的回合交替机制。 2. 状态机管理： 能够清晰区分游戏状态（主菜单、游戏运行、物品栏、游戏结束）。
战斗/交互	1. 基础移动： 支持八方向移动。 2. 战斗系统： 支持近战攻击，根据属性（HP, Def, Pow）进行伤害计算。 3. 物品系统： 支持拾取、存放在物品栏、使用（如药水）和丢弃。
界面/I/O	1. 控制台渲染： 使用 ASCII 字符或扩展字符集渲染地图和实体。 2. 用户输入： 响应键盘输入（移动、菜单选择）。

4.1.2. 界面交互逻辑

状态 (RunState)	界面显示内容	交互逻辑
MainMenu	显示“开始新游戏”、“加载游戏”、“退出”选项。	玩家通过上下箭头选择， Enter 键确认，切换到 NewGame 或 LoadGame 状态。
NewGame	游戏初始化（地图生成、实体创建）。	自动转换为 AwaitingInput 状态。
AwaitingInput	渲染地图、实体、角色状态栏和消息日志。	核心逻辑： 等待玩家输入移动或操作指令。若收到指令，转换为 PlayerTurn 。
PlayerTurn	界面不变。	系统逻辑： 运行玩家的移动、战斗等系统。完成后，自动转换为 MonsterTurn 。
MonsterTurn	界面不变。	系统逻辑： 运行所有怪物的 AI、移动和战斗系统。完成后，自动转换为 AwaitingInput 。
ShowInventory	渲染物品栏菜单。	玩家可以按键使用或丢弃物品，或退出菜单返回 AwaitingInput 。
GameOver	显示“ Game Over ”信息。	玩家按任意键返回 MainMenu 。



游戏基本流程图

（二）项目创建与环境配置

4.2.1. 项目创建记录

使用 Rust 的包管理器 Cargo 创建项目骨架：

```
cargo new my_roguelike_ecs
cd my_roguelike_ecs
```

4.2.2. Cargo.toml 依赖配置

根据需求分析，配置项目所依赖的核心库。核心依赖包括 ECS 框架 (specs) 和控制台渲染库 (rltk)

Cargo.toml 配置清单：

```
[dependencies]
rltk = { version = "0.8.0", features = ["serde"] }
specs = { version = "0.17", features = ["serde"] }
specs-derive = "0.4.1"
serde = { version = "^1.0.44", features = ["derive"] }
serde_json = "^1.0.44"
```

RLTK（Rust-Like ToolKit）用于控制台渲染和输入处理，SPECS 用于创建 ECS 框架以及简化组件函数，SERDE 以及 SERDE_JSON 用于对数据转为 JSON 格式保存的序列化以及反序列化。

4.2.3. 主程序结构

main.rs 用于初始化 RLTK 窗口并配置 ECS 世界：

ECS World 初始化： 创建 World 实例，注册所有必需的组件和资源。

RLTK 引擎配置： 设置控制台的尺寸、标题和字体。

游戏状态机： 定义并初始化 GameState 结构体，并实现 rltk::GameState Trait。

初始状态： 设置初始 RunState 为 MainMenu，进入主循环。

每个步骤代码如下：

1. 定义游戏状态机结构体

```
pub struct State { pub ecs: World }
```

2. 实现 RLTK 的 GameState Trait

```
impl GameState for State {
    fn tick(&mut self, ctx: &mut rltk::Rltk) {
        // ... 在这里实现游戏状态 RunState 的逻辑切换 }
    }
}
```

3. RLTK 初始化

```
let context = RltkBuilder::simple80x50().with_title("Rust Roguelike (ECS)").build()?;
```

4. ECS World 初始化

```
let mut gs = State {   ecs: World::new()   };
```

5. 注册组件 (Component) 和资源 (Resource)

```
gs.ecs.register::<Position>();
```

```
gs.ecs.insert(RunState::MainMenu);
```

6. 启动主循环

```
rltk::main_loop(context, gs)
```

（三）界面布局设计

4.3.1.控制台分层渲染

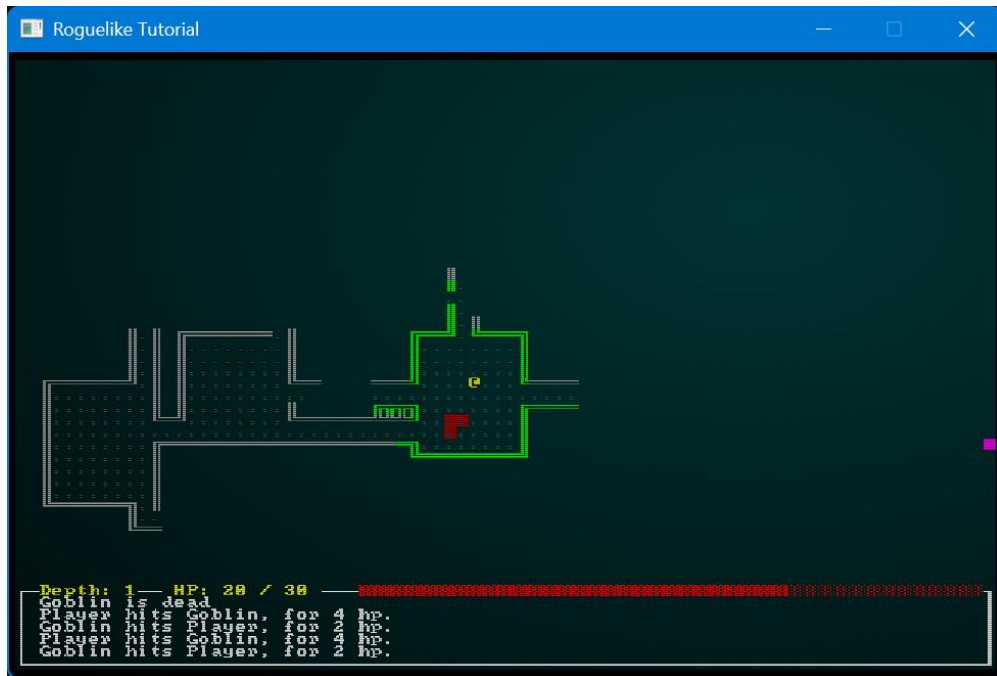
为了实现复杂的 UI 叠加，RLTK 支持多层控制台的概念。每一层都是一个独立的缓冲区，可以设置不同的字符、颜色和透明度，然后按照堆栈顺序叠加渲染。

层级设计：

Layer 0 (Map): 地图瓦片、地板、墙壁。

Layer 1 (Entities): 玩家、怪物、物品、陷阱。

Layer 2 (UI/HUD): 消息日志、生命值条、状态效果图标。



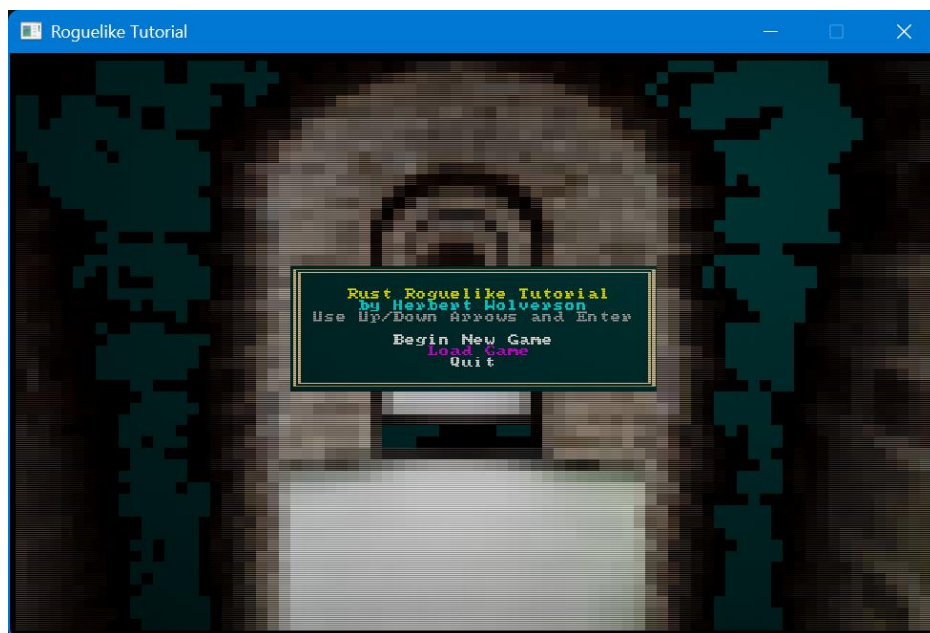
游戏运行时的控制台分层渲染效果图

4.3.2.渲染管线与双缓冲技术

RLTK 采用标准的双缓冲 (Double Buffering) 机制来消除屏幕绘制时的闪烁或撕裂现象。

绘制阶段: 所有系统（特别是 `RenderSystem`）将输出绘制到后台缓冲区。

交换阶段: 在 `MainLoop` 的末尾，两个缓冲区交换角色，后台内容瞬间显示到屏幕上。



启动时的初始界面

4.3.3. 输入事件处理与意图组件

RLTK 的输入处理将原始键盘事件转化为 ECS 能够消费的意图组件，从而解耦了输入源和逻辑处理。

输入到组件: 当玩家按下 'W' 键，`PlayerInputSystem` 捕获该事件，并创建一个 `WantsToMove{x: -1, y: 0}` 组件，将其附加到玩家实体上。

系统响应: 在接下来的 `PlayerTurn` 状态中，`MovementSystem` 仅查询具有 `WantsToMove` 组件的实体并执行移动逻辑。

（四）界面样式设计

本项目采用配色方案旨在提供清晰的对比度、营造暗黑地牢的氛围，同时利用色彩标记关键信息。

配色方案如下:



游戏配色展示

基础调色板: 采用深黑 (`rltk::BLACK`) 作为背景主色，突出控制台游戏的经典风格。

地图记忆色: 对于已被发现但当前不可见的瓦片，使用低饱和度的灰度色进行渲染 (`rltk::DARK_GRAY` 或 `rltk::BLUE` 的阴影)，以模拟“地图记忆”的视觉效果。

活动实体色: 玩家和怪物使用高亮、高对比度的颜色（如玩家使用白色/黄色，怪物使用红色/绿色），确保它们在地图背景中具有最高优先级。

UI/高亮色: 状态栏和菜单中的关键交互元素（如菜单选中项）使用洋红色 (`rltk::MAGENTA`) 或青色 (`rltk::CYAN`) 进行突出显示，如 3.2.1 节所述。

字符集 (Glyphs): 使用 ASCII 或 Code Page 437 字符集，例如 `#` 代表墙壁，`.` 代表地板，`@` 代表玩家。字符的颜色直接通过 `Renderable` 组件控制。

（五）交互设计实现

本项目的交互设计严格遵循回合制和指令驱动的 Roguelike 范式，核心机制基于状态机和意图组件。

游戏只在 `RunState::AwaitingInput` 状态下接受玩家输入。

玩家输入后，状态立即切换到 `RunState::PlayerTurn`，然后依次执行所有逻辑系统，直到切换到 `RunState::MonsterTurn`，最终返回 `RunState::AwaitingInput`。

这种设计确保了交互的原子性：一次输入，一个回合。使用键盘方向键或 WASD/HJKL 键进行移动。RLTK 捕获这些键位并由 `PlayerInputSystem` 转换为 `WantsToMove` 组件。在主游戏状态按下 'i' 键会切换到 `RunState::ShowInventory`，触发库存系统的渲染和交互逻辑。



交互反馈主要通过消息日志（通知玩家行动结果）和实体高亮（通过改变 `Renderable` 组件的颜色来高亮被攻击或被选中的实体）来实现。

五、实验结果

（一）地牢生成与空间结构算法

5.1.1. Map 资源的数据结构设计

地图数据结构 Map 是 ECS 世界中的单例资源 (Resource)，是所有空间操作的基础。

数组名称	类型	存储结构与技术点	作用与索引
tiles	Vec<TileType>	采用一维扁平化数组（Row-Major Order），通过 map.xy_idx(x, y) 映射到 $y * Width + x$ 。此结构优化了数据局部性，有利于 CPU 缓存命中。	存储每个坐标的瓦片类型（Wall, Floor, DownStairs）。
revealed_tiles	Vec<bool>	同样是扁平化数组，用于存储玩家是否“发现”过某个瓦片。	实现地图记忆功能。
visible_tiles	Vec<bool>	存储当前回合哪些瓦片在玩家的 FOV (Field of View) 内。	实时渲染，实现“战争迷雾”效果。
blocked	Vec<bool>	由 MapIndexingSystem 实时维护的布尔数组。	A* 寻路算法的输入，快速判断一个瓦片是否可通行（包括墙壁和不可移动的实体）。
tile_content	Vec<Vec<Entity>>	地图内容索引。每个索引对应一个瓦片，存储着当前瓦片上所有实体 ID (Entity) 的列表。	实现高效的空间查询，如：判断目标位置是否有怪物、拾取物品。

5.1.2. 房间与走廊生成 (Rooms and Corridors)

本项目采用非递归的房间与走廊连接算法，专注于生成开放且连贯的地牢结构。

房间生成与放置：

- 1.首先，在地图上随机尝试放置一系列矩形房间 (Rect)。
- 2.碰撞检测是核心：新房间必须保证与所有已放置的房间都不发生重叠。
- 3.生成成功后，将房间内部的瓦片设置为 Floor，外部设置为 Wall。

走廊连接：

- 1.遍历已生成的房间列表，依次连接相邻房间。
- 2.连接采用 L 形走廊：从房间 A 的中心点 (Ax, Ay) 开始，到达房间 B 的中心点 (Bx, By)，分两段直线绘制。

第一段：从 (Ax, Ay) 移动到 (Ax, By)（垂直或水平）。

第二段：从 (Ax, By) 移动到 (Bx, By)（水平或垂直）。
- 3.每段直线都通过循环迭代绘制 Floor 瓦片，保证走廊路径被添加到地图中。

5.1.3. 视野计算

视野计算系统是游戏沉浸感的来源。采用 Recursive Shadowcasting 算法，通过模拟光线投射，产生平滑、真实的视野。

阶段	机制描述	技术细节
初始化	每个回合开始时，清空玩家 Viewshed 组件中的 visible_tiles 列表，并将全局 Map 资源中的 visible_tiles 数组重置为 false。	确保视野是实时的，不保留上一回合的状态。
递归投射	从玩家实体的位置开始，向 8 个方向（或更多方向）进行扇形投射。算法核心在于对斜坡和角度的递归计算。	当光线遇到不透明的瓦片（如墙壁）时，算法会调整随后的投射角度（递归调用），模拟阴影的产生，确保视野边缘的平滑。
瓦片记忆	在投射过程中，任何被标记为 visible_tiles 为 true 的瓦片，其对应的 revealed_tiles 也被设置为 true。	允许玩家离开一个区域后，该区域的地图结构仍然可见（只是没有实体）。
可见性更新	遍历地图上的所有实体。如果一个实体的 Position 位于 visible_tiles 范围内，则渲染该实体；否则不渲染。	这是实现“战争迷雾”和隐藏怪物的基础。

（二）核心游戏系统与行为实现

5.2.1. 移动系统 (MovementSystem)

移动系统的核心在于意图处理和有效性验证。它在玩家输入系统 (player.rs) 生成 WantsToMove 意图后运行。

步骤	详细逻辑	涉及组件/资源
意图查询	迭代所有带有 WantsToMove 组件的实体。	WantsToMove, Position, Entities
有效性检查	边界/地图阻挡: 检查目标坐标是否在地图范围内, 并且目标瓦片 (TileType) 必须是 Floor。	Map 资源
实体阻挡检查	通过 Map 资源的 blocked 数组和 tile_content 索引, 检查目标位置是否已被其他具有 BlocksTile 组件的实体占据。	BlocksTile, Map 资源
意图转换 (战斗)	如果目标位置被一个非阻挡 (即可攻击) 的实体占据, 且该实体具有 CombatStats 组件, 则: 1. 移除 WantsToMove 组件。 2. 插入 WantsToMelee 组件 (攻击意图)。	WantsToMove, WantsToMelee
位置更新	如果移动合法且未触发战斗, 则更新实体的 Position 组件。	Position
清理	移除所有已处理的 WantsToMove 组件。	WantsToMove

5.2.2. 怪物 AI 与寻路系统 (MonsterTurnSystem)

怪物 AI 系统是复杂的, 必须在有限的回合时间内高效运行。

AI 行为状态机:

- 1.追逐 (Chase): 当玩家的 Position 在怪物的 Viewshed 内时, AI 转换为追逐状态。
- 2.寻路集成 (A*): 怪物使用 A* 算法计算从自身到玩家的最短路径。Map 资源的 blocked 数组作为寻路的成本图输入。
- 3.攻击触发: 如果 A* 算法的下一步路径点与玩家的 Position 相邻, AI 系统将跳过 WantsToMove 意图, 直接插入 WantsToMelee 意图, 触发近战。

AI 伤害应用 (饥饿系统联动):

在 hunger_system.rs 中可见, 饥饿系统 (HungerSystem) 不仅处理玩家的饥饿状态, 其设计也允许理论上给其他具有 HungerClock 的实体应用饥饿伤害。

饥饿的终极状态是 `HungerState::Starving`，该状态下，系统会直接使用 `SufferDamage::new_damage()` 方法向实体（如果是玩家，则有日志提示）添加 1 点伤害。这体现了 ECS 系统对任何实体的通用伤害应用能力。

5.2.3. 战斗与伤害处理系统

战斗逻辑严格遵循 ECS 模式，从意图到伤害结果被分解为两个系统。

1. 意图生成 (`MeleeCombatSystem`):

数据查询：系统迭代 `WantsToMelee`，读取攻击者和目标的 `CombatStats`, `Name`, `Position`。

装备加成计算 (Bonus Integration): 战斗系统会额外遍历所有具有 `Equipped` 组件的实体，检查其是否拥有 `MeleePowerBonus` 或 `DefenseBonus`。

$$\text{Total Power} = \text{Base Power} + \sum \text{MeleePowerBonus}$$

$$\text{Total Defense} = \text{Base Defense} + \sum \text{DefenseBonus}$$

伤害计算：原始伤害值是总攻击力减去总防御力的结果，并确保伤害值 ≥ 0 。

$$\text{Damage} = \max(0, \text{Total Power} - \text{Total Defense})$$

伤害意图：如果伤害 > 0 ，则向目标实体插入 `SufferDamage` 组件，携带计算出的伤害值。

粒子系统集成：在目标位置插入粒子系统意图 (`ParticleBuilder::request`)，创建视觉反馈（例如橙色的 !! 字符）。

2. 伤害应用 (`DamageSystem`):

应用伤害：迭代所有具有 `SufferDamage` 组件的实体，将所有伤害值求和后，从实体的 `CombatStats::hp` 中减去。

血迹记录：伤害发生后，系统会在实体所在位置的 `Map::bloodstains` 集合中插入血迹标记，用于渲染时的视觉效果。

生命周期管理 (delete_the_dead): 在 DamageSystem 运行后，一个辅助函数会检查所有 $HP \leq 0$ 的实体。

玩家死亡： 游戏状态 (RunState) 切换为 GameOver。

怪物死亡： 向其添加 WantsToDie 组件，等待后续清理系统将其从世界中移除 (ecs.delete_entity(victim))。

5.2.4. 魔法道具与物品交互系统

物品交互逻辑集中在 inventory_system.rs 中的几个系统内，ItemUseSystem 负责处理所有物品的使用效果，特别是针对定义的魔法组件。

魔法/效果组件	所属系统	意图触发	核心逻辑实现
ProvidesFood	ItemUseSystem	触发时，物品是食物。	获取玩家实体的 HungerClock 组件。 执行逻辑：重置 duration 值，并可能将 HungerState 升级为 WellFed，消除饥饿带来的惩罚（如降低战斗力）。
MagicMapper	ItemUseSystem	触发时，物品是“地图全开卷轴”。	关键： 直接获取全局资源 Map 的可写引用 (WriteExpect<'a, Map>)。 执行逻辑：遍历 Map::revealed_tiles 数组，将所有值设置为 true。
ProvidesHealing	ItemUseSystem	触发时，物品是治疗药水。	查找玩家的 CombatStats，将 hp 增加（限制在 max_hp 内），并记录日志。

通过将魔法效果定义为简单的标记组件（如 MagicMapper），并让核心系统（ItemUseSystem）根据实体所拥有的标记来执行相应的全局或局部操作，极大地增强了系统的可扩展性和解耦性。

5.2.5. 物品系统与交互

物品系统通过组件 (Component) 的组合实现物品的定义、存储、装备和使用。

1. 物品定义组件 (components.rs)

组件名称	类型	描述
Item	空标记	标记该实体是一个可交互物品。
InBackpack	pub owner : Entity	标记物品当前在某个实体（owner）的背包中，脱离地图。

组件名称	类型	描述
Equippable	pub slot : EquipmentSlot	标记物品可装备，并指定装备槽位（Melee 或 Shield）。
Equipped	owner : Entity, slot : EquipmentSlot	标记物品已被 owner 实体装备到指定槽位。
MeleePowerBonus	pub power : i32	增加角色的近战攻击力（如武器）。
DefenseBonus	pub defense : i32	增加角色的防御力（如盔甲、盾牌）。
Consumable	空标记	标记物品使用后会从游戏中销毁（如药水、卷轴）。

2. 物品交互系统 (inventory_system.rs)

交互逻辑通过四个独立的系统和意图组件实现：

ItemCollectionSystem (拾取):

意图：WantsToPickupItem。

逻辑：移除物品的 Position 组件，并插入 InBackpack 组件。

ItemDropSystem (丢弃):

意图：WantsToDropItem。

逻辑：移除 InBackpack 组件，并将物品的 Position 设置为丢弃者的位置。

ItemRemoveSystem (卸下装备):

意图：WantsToRemoveItem。

逻辑：移除物品的 Equipped 组件，并重新插入 InBackpack 组件，将物品放回背包。

ItemUseSystem (使用/装备):

意图：WantsToUseItem。

逻辑：核心处理单元。根据目标物品是否带有 Equippable (装备/替换)，ProvidesHealing (治疗)，ProvidesFood (饱食)，MagicMapper (全图显示) 等组件，执行相应的效果。使用后，如果物品带有 Consumable 标记，则销毁物品实体。

5.2.6. 游戏存档与加载系统 (SaveLoadSystem)

游戏存档功能被封装在 `saveload_system.rs` 模块中，是基于 `specs::saveload` 特性实现的。

1. 序列化标记 (SerializeMe):

为了控制哪些实体需要被保存，项目定义了一个特殊的标记组件 `SimpleMarker<SerializeMe>`。

机制：只有拥有这个标记组件的实体才会被序列化。在游戏初始化时，所有核心实体（如玩家、怪物、地图上的物品）都会被赋予此标记。

2. 存档流程 (save_game):

RunState 检查： 确保当前 `RunState` 为 `SaveGame`。

数据收集： 使用 `specs::saveload::MarkedBuilder` 收集所有标记为 `SerializeMe` 的实体及其组件数据。

序列化： 使用 `serde` 库（通过 `serde_json` 或其他格式）将 `ECS World` 的数据结构（包括所有组件、资源）序列化为一个字符串。

写入文件： 将序列化后的字符串写入本地文件系统（通常是 `target/savegame.json` 或类似路径）。

状态切换： 存档成功后，将 `RunState` 切换回 `MainMenu`。

3. 加载流程 (load_game):

检查存档： 检查存档文件是否存在。

读取文件： 从文件中读取序列化的字符串。

反序列化： 使用 `specs::saveload::DeserializeComponents` 将数据反序列化回 `ECS World` 的结构。

重建世界： 清理当前的 `ECS World`，并用加载的数据重建所有实体和资源。

状态切换： 加载成功后，将 `RunState` 切换到 `PreRun`，准备开始游戏。

通过将所有核心逻辑和数据都封装在 `ECS` 组件和资源中，存档和加载系统能够高效地对整个游戏状态进行快照和恢复。